

State Machine Generator

User Manual

System Documentation

March 3, 2026

Contents

1	Overview	2
2	Generating the Code	2
2.1	Command Line Options	2
2.2	Outputs	2
3	SMB File Structure	3
3.1	Language-Specific Syntax	3
4	Integration & API	4
4.1	Rust Integration	4
4.2	C Integration	4
4.3	Python Integration	5
4.4	TypeScript Integration	6
4.5	Context Struct	7
5	Writing Logic: Handlers vs. Hooks	7
5.1	Local Handlers (Per State/Transition)	7
5.2	Global Hooks (System-wide)	7
5.3	Available Variables in Code Blocks	7
6	Transitions	8
6.1	Basic Transition Syntax	8
6.2	Transition Execution Order	8
6.3	Using Actions (The Mealy Aspect)	9
6.4	Path Syntax	9
6.5	Transition Scenarios	9
6.6	Termination	10
7	History	10
8	Orthogonal States (Parallelism)	10
8.1	Safety: <code>transition_fired</code>	11
8.2	Fork Transitions (Drilling Down)	11
8.3	Cross-Limb Transitions	11
8.4	Joining (Synchronization)	11

9	Decisions (Logic Trees)	12
9.1	Defining Decisions	12
9.2	Using Decisions	12
10	Complete Example	13

1 Overview

This tool generates a safety-critical Hierarchical State Machine (HSM) from a YAML/SMB definition file. It supports four output languages — **Rust**, **C**, **Python**, and **TypeScript** — and includes advanced features such as orthogonal regions (parallelism), history patterns, decision trees, and fork transitions.

The generator also produces a **Graphviz DOT** file for visualizing the state machine diagram.

2 Generating the Code

The generator is a Python script (`sm-compiler.py`) that validates your model and produces code in the selected target language.

```
# Language is read from the 'language:' key in the .smb file
uv run python sm-compiler.py model.smb

# Override the language
uv run python sm-compiler.py model.smb --lang rust
uv run python sm-compiler.py model.smb --lang c
uv run python sm-compiler.py model.smb --lang typescript

# Custom output base path (extensions added automatically)
# Produces build/myfsm.rs and build/myfsm.dot
uv run python sm-compiler.py model.smb -o build/myfsm
```

Listing 1: Command Line Usage

2.1 Command Line Options

Option	Default	Description
file	(required)	Path to the input <code>.smb</code> file.
--lang	from file	Output language: <code>rust</code> , <code>c</code> , <code>python</code> , or <code>typescript</code> . If omitted, reads the <code>language:</code> key from the SMB file (defaults to <code>rust</code> if absent). The <code>language:</code> key may be a list, in which case code is generated for all listed languages.
-o, --output	./statemachine	Output base path without extension. Language-specific extensions (<code>.rs</code> , <code>.c</code> , <code>.h</code> , <code>.py</code> , <code>.ts</code> , <code>.dot</code>) are appended automatically. Parent directories are created if needed.

2.2 Outputs

Given `-o path/to/name`, the following files are produced:

Language	Output Files	Description
Rust	<code>name.rs</code>	Complete Rust implementation (single file).
C	<code>name.h</code> , <code>name.c</code>	Header with struct/function declarations and source with implementation. The <code>.c</code> file includes <code>name.h</code> automatically.

Language	Output Files	Description
Python	name.py	Complete Python implementation (single file). Import with <code>from name import StateMachine</code> .
TypeScript	name.ts	Complete TypeScript implementation (single file). Import with <code>import { StateMachine } from "../name"</code> .
All	name.dot	Graphviz visualization of the state machine.

3 SMB File Structure

The input file uses YAML syntax with the extension `.smb`. A complete model has this top-level structure:

```
language: rust                # Target language (rust, c, python, or
                             typescript)

includes: |                   # Code placed before the Context struct
  // imports, helper functions, etc.

context: |                    # User-defined fields for the Context struct
  pub counter: i32,           # (Rust syntax)
  pub flag: bool,

context_init: |               # Initialization code for context fields
  counter: 0,                 # (Rust: struct init syntax)
  flag: true,

hooks:                        # Global code injected into all states
  entry: |
    // runs on every state entry
  exit: |
    // runs on every state exit
  do: |
    // runs on every state tick
  transition: |
    // runs on every transition

initial: first_state          # Name of the initial child state

states:
  first_state:
    # ... state definition ...
```

Listing 2: Top-level SMB structure

3.1 Language-Specific Syntax

Since `includes`, `context`, `context_init`, guards, and actions contain raw target-language code, they must use the syntax of the selected output language:

Element	Rust	C	Python	TypeScript
Context access	<code>ctx.field</code>	<code>ctx->field</code>	<code>ctx.field</code>	<code>ctx.field</code>
Context fields	<code>pub field: Type,</code>	<code>Type field;</code>	<code>field: Type</code>	<code>field: Type = val;</code>
Context init	<code>field: value,</code>	<code>sm->ctx.field = val;</code>	<code>ctx.field = val</code>	<code>ctx.field = val;</code>

Element	Rust	C	Python	TypeScript
Print	<code>println!("{}", ...)</code>	<code>printf("...\n")</code>	<code>print("...")</code>	<code>console.log("...")</code>
String type	<code>&str</code>	<code>const char*</code>	<code>str</code>	<code>string</code>
Boolean	<code>bool</code>	<code>bool</code>	<code>bool</code>	<code>boolean</code>
True/False	<code>true/false</code>	<code>true/false</code>	<code>True/False</code>	<code>true/false</code>
Logical OR	<code> </code>	<code> </code>	<code>or</code>	<code> </code>
Logical AND	<code>&&</code>	<code>&&</code>	<code>and</code>	<code>&&</code>
Equality	<code>==</code>	<code>==</code>	<code>==</code>	<code>===</code>

4 Integration & API

4.1 Rust Integration

The generated code provides a `StateMachine` struct. You must initialize it and call `tick()` cyclically.

```
mod statemachine;
use statemachine::StateMachine;

fn main() {
    let mut sm = StateMachine::new();

    while sm.is_running() {
        sm.ctx.now += 0.1; // Advance logical time
        sm.tick();
        println!("{}", sm.get_state_str());
        std::thread::sleep(std::time::Duration::from_millis(100));
    }
}
```

Listing 3: Example Rust driver (main.rs)

Rust API

Function	Location	Description
<code>StateMachine::new()</code>	constructor	Creates and starts the state machine.
<code>sm.tick()</code>	sm	Executes one cycle of logic (do-activities & transitions).
<code>sm.is_running()</code>	sm	Returns <code>false</code> if the machine has terminated.
<code>sm.get_state_str()</code>	sm	Returns a string like <code>/run/active</code> or <code>/p/[a/x,b/y]</code> .
<code>ctx.in_state_X()</code>	ctx	Returns <code>true</code> if state X is currently active.

4.2 C Integration

The generated code provides a `StateMachine` struct with an `SM_Context` inside. Use the provided API functions.

Listing 4: Example C driver (main.c)

```
#include "statemachine.h"
#include <stdio.h>
#include <unistd.h>
```

```

int main(void) {
    StateMachine sm;
    sm_init(&sm);

    char state_buf[256];

    while (sm_is_running(&sm)) {
        sm_tick(&sm);
        sm_get_state_str(&sm, state_buf, sizeof(state_buf));
        printf("State: %s\n", state_buf);
        usleep(100000);
    }

    return 0;
}

```

Compile with:

```
gcc main.c statemachine.c -o my_program
```

C API

Function	Description
<code>sm_init(StateMachine* sm)</code>	Initializes context (zeroes memory, applies initial state, enters initial state.
<code>sm_tick(StateMachine* sm)</code>	Executes one cycle of logic.
<code>sm_is_running(StateMachine* sm)</code>	Returns true if the machine has not terminated.
<code>sm_get_state_str(StateMachine* sm, char* buf, size_t max)</code>	Writes the current state path into buf.
<code>IN_STATE_X</code>	Macro: evaluates to true if state X is current. Used inside guards and actions where ctx is in scope.

4.3 Python Integration

The generated code provides a `StateMachine` class. No compilation needed — run directly with Python 3.

Listing 5: Example Python driver

```

from statemachine import StateMachine

sm = StateMachine()
print(sm.get_state_str())

while sm.is_running():
    sm.tick()
    sm.ctx.counter += 1
    print(f"{sm.ctx.counter:02d}: {sm.get_state_str()}")

```

Run with:

```
python3 driver.py
```

Python API

Method / Attribute	Description
<code>StateMachine()</code>	Creates and initializes the state machine. Enters the initial state.
<code>sm.tick()</code>	Executes one cycle of logic.
<code>sm.is_running()</code>	Returns <code>True</code> if the machine has not terminated.
<code>sm.get_state_str()</code>	Returns the current state path as a string.
<code>sm.ctx</code>	Direct access to the <code>Context</code> object (read/write fields).
<code>ctx.in_state_X()</code>	Returns <code>True</code> if state <code>X</code> is currently active.

Note: Python uses `None` for empty function pointers and first-class functions for state callbacks. The `context_init` code uses `ctx.field = value` syntax, where `ctx` is an alias for `self` inside the `Context.__init__` method.

4.4 TypeScript Integration

The generated code provides a `StateMachine` class exported from a single `.ts` file. No compilation step is needed — run with `npm run tsx` or import as a module.

```
import { StateMachine } from './statemachine';

const sm = new StateMachine();
console.log(sm.getStateStr());

while (sm.isRunning()) {
  sm.tick();
  sm.ctx.counter += 1;
  console.log(`${String(sm.ctx.counter).padStart(2, '0')}: ${sm.getStateStr()}`);
}
```

Listing 6: Example TypeScript driver

Run with:

```
npm run tsx driver.ts
```

TypeScript API

Method / Attribute	Description
<code>new StateMachine()</code>	Creates and initializes the state machine. Enters the initial state.
<code>sm.tick()</code>	Executes one cycle of logic.
<code>sm.isRunning()</code>	Returns <code>true</code> if the machine has not terminated.
<code>sm.getStateStr()</code>	Returns the current state path as a string.
<code>sm.ctx</code>	Direct access to the <code>Context</code> object (read/write fields).
<code>ctx.inState_X()</code>	Returns <code>true</code> if state <code>X</code> is currently active.

Note: TypeScript uses `null` for empty function pointers and first-class functions for state callbacks. Function pointer fields are typed as `StateFn | null`. The `context_init` code uses `ctx.field = value;` syntax, where `ctx` is an alias for `this` inside the `Context` constructor. Use `===` for equality checks in guards.

4.5 Context Struct

All backends generate a `Context` struct or class (Rust: `Context`, C: `SM_Context`, Python/TypeScript: `Context`) containing:

- `now`: Logical time (`f64/double`). You update this before each tick.
- `state_timers[]`: Per-state entry timestamps (set automatically on entry).
- `transition_fired`: Safety flag for orthogonal regions (managed automatically).
- `terminated`: Set when a `null` transition fires (managed automatically).
- Hierarchy pointers: Function pointers tracking the active state at each level (managed automatically).
- User fields: Whatever you declare in the `context:` block.

5 Writing Logic: Handlers vs. Hooks

5.1 Local Handlers (Per State/Transition)

These are defined inside specific states or transitions in the YAML. They apply only to that specific element.

- `entry`: Runs once when entering the specific state.
- `do`: Runs every tick while in the specific state.
- `exit`: Runs once when leaving the specific state.
- `action`: Runs during a specific transition (see Section 6).

5.2 Global Hooks (System-wide)

Hooks are defined at the root level under the `hooks:` key. They inject code into **every** occurrence of an event type across the entire machine (e.g., for logging).

Hook Key	Trigger Event
<code>hooks: entry</code>	Injected into all states' entry handlers.
<code>hooks: do</code>	Injected into all states' do handlers.
<code>hooks: exit</code>	Injected into all states' exit handlers.
<code>hooks: transition</code>	Injected into all transitions (equivalent to a global action).

Note: There is no `hooks: action`. Use `hooks: transition` instead.

5.3 Available Variables in Code Blocks

The following variables are injected into the scope of your code snippets. Use the appropriate syntax for your target language.

Variable	Scope	Description
<code>ctx</code>	All	The shared context struct. Rust: <code>&mut Context</code> . C: <code>SM_Context*</code> . Python/TypeScript: <code>Context</code> instance.
<code>time</code>	All	Time elapsed (seconds) since entering the current state (<code>f64/double</code>).
<code>state_name</code>	All	Short name of the current state (e.g., <code>"idle"</code>).
<code>state_full_name</code>	All	Full path of the current state (e.g., <code>"/run/idle"</code>).
<code>t_src</code>	Transitions	The full path of the source state.
<code>t_dst</code>	Transitions	The full path/description of the target.

6 Transitions

A transition determines how the machine moves between states.

6.1 Basic Transition Syntax

```

transitions:
- guard: ctx.counter == 5      # Condition (target language expression)
  action: |                    # Code executed during the transition
    // runs BEFORE exit/entry sequence
  to: target_state             # Where to go

```

Listing 7: Transition syntax

All three fields are optional:

- Omitting `guard` makes the transition unconditional (`true`).
- Omitting `action` means no transition-time code.
- `to: null` triggers machine termination.

6.2 Transition Execution Order

When a transition fires:

1. Evaluate the `guard` condition.
2. Set `transition_fired = true`.
3. Execute `action` code (if any).
4. Execute the exit sequence (from current state up to the Least Common Ancestor, bottom-up).
5. Execute the entry sequence (from LCA down to target state, top-down).

6.3 Using Actions (The Mealy Aspect)

The `action` keyword allows you to execute code *during* the transition, before the old state is exited.

```
states:
  reading:
    transitions:
      - guard: ctx.value > 100
        action: |
          println!("Moving_{}_->{}", t_src, t_dst);
          ctx.log_alert("Threshold_Exceeded");
        to: alert_mode
```

Listing 8: Rust example using action variables

```
states:
  reading:
    transitions:
      - guard: ctx->value > 100
        action: |
          printf("Moving_%s->%s\n", t_src, t_dst);
          log_alert(ctx, "Threshold_Exceeded");
        to: alert_mode
```

Listing 9: C example using action variables

6.4 Path Syntax

The `to:` keyword supports several addressing modes:

Syntax	Description
/absolute/path	From the root. Use for long jumps across the hierarchy.
sibling	A state in the same parent (lateral transition).
./child	A direct child of the current state (drill down).
../uncle	Up one level, then a sibling of the parent.
.	Self-transition: the state exits and immediately re-enters.
null	Machine termination. Exits all states up to root and stops.
@decision_name	Delegate to a named decision tree (see Section 9).
/path/[a/tgt, b/tgt]	Explicit fork into an orthogonal state (see Section 8).

6.5 Transition Scenarios

Lateral (Sibling to Sibling)

- Exits: A
- Enters: B

Drill Down (Parent to Child)

- Exits: None (P remains active).
- Enters: C

Move Up (Child to Parent/Uncle)

- **Exits:** C, then Parent P.
- **Enters:** U

6.6 Termination

Setting `to: null` causes the machine to exit all states from the current state up through root, then stop. After termination, `is_running()` (Rust/Python), `isRunning()` (TypeScript), or `sm_is_running()` (C) returns false.

7 History

A composite state with `history: true` remembers its last active child. When re-entered, it resumes from that child instead of the `initial` state.

```
process:
  history: true
  initial: step_a
  states:
    step_a:
      transitions:
        - guard: ctx.counter == 3
          to: step_b
    step_b:
      transitions:
        - guard: ctx.counter == 5
          to: /other_state
```

Listing 10: History example

If the machine leaves `process` while in `step_b`, and later transitions back to `process`, it will re-enter `step_b` directly (skipping `step_a`).

8 Orthogonal States (Parallelism)

An orthogonal state runs multiple child regions simultaneously.

Keyword: `orthogonal: true`

```
parallel_task:
  orthogonal: true
  states:
    region_a:
      initial: idle_a
      states:
        idle_a: {}
        working_a: {}
    region_b:
      initial: idle_b
      states:
        idle_b: {}
        working_b: {}
```

Listing 11: Orthogonal state definition

When `parallel_task` is entered, **all** child regions are entered simultaneously. Each region ticks independently. When exiting, all regions are exited.

The state path for orthogonal states uses bracket notation: `/parallel_task/[region_a/idle_a,region_b/]`

8.1 Safety: `transition_fired`

When a transition fires in one region, the other regions' ticks are skipped for that cycle to prevent inconsistent state. This is handled automatically via the `transition_fired` flag.

8.2 Fork Transitions (Drilling Down)

You can enter an orthogonal state by targeting its children directly.

1. **Implicit Fork:** `to: /run/g/a/sub_state`

The generator forces region `a` to `sub_state`. Other regions start at their `initial` states.

2. **Explicit Fork:** `to: /run/g/[a/sub_state, b/other]`

Explicitly sets the target for multiple regions at once.

8.3 Cross-Limb Transitions

A transition from one region to another region within the same orthogonal parent is a **cross-limb transition**. The generator handles this by exiting the target region's current state and entering the new target, while the source region remains unaffected.

8.4 Joining (Synchronization)

To “join” parallel states, place a transition on the **parent** that checks the status of the **children**.

```
main_task:
  orthogonal: true
  states:
    A:
      initial: working
      states:
        working:
          transitions:
            - guard: ctx.done_a
              to: finished
        finished: {}
    B:
      initial: working
      states:
        working:
          transitions:
            - guard: ctx.done_b
              to: finished
        finished: {}

# Transition on the parent container
transitions:
  - guard: IN_STATE(root_main_task_A_finished) && IN_STATE(
    root_main_task_B_finished)
    to: next_step
```

Listing 12: Join pattern (Rust guards)

Note: Use the `IN_STATE(X)` macro in guards for portable state checks. In Rust and Python, it expands to `ctx.in_state_X()`. In TypeScript, it expands to `ctx.inState_X()`. In C, it expands to the `IN_STATE_X` macro.

9 Decisions (Logic Trees)

Decisions allow you to externalize branching logic. A decision node can point to states, or **chain to other decisions**.

Restriction: You cannot use `action` or `hooks` inside a decision block. Actions must be placed on the transition *leading to* the decision, or on the state transitions following it.

9.1 Defining Decisions

Decisions can be defined at the root level or within any state. They are collected into a flat namespace, so names must be unique across the entire model.

```
decisions:
  validate_input:
    - guard: ctx.input < 0
      to: /error/negative
    - guard: ctx.input > 100
      to: "@check_permissions"      # Chain to another decision
    - to: /process/calc             # Default (no guard)

  check_permissions:
    - guard: ctx.is_admin
      to: /admin/override
    - to: /error/denied
```

Listing 13: Decision tree definition

9.2 Using Decisions

Reference a decision with the `@` prefix in the `to:` field:

```
transitions:
- guard: ctx.data_ready
  to: "@validate_input"
```

Listing 14: Using a decision in a transition

Decisions can also be defined locally within a state:

```
my_state:
  decisions:
    local_check:
      - guard: ctx.x > 0
        to: positive
      - to: negative
  initial: waiting
  states:
    waiting:
      transitions:
        - to: "@local_check"
    positive: {}
    negative: {}
```

Listing 15: Local decision definition

10 Complete Example

Below is a minimal but complete SMB file demonstrating the key features, with Rust as the target language.

```
language: rust

includes: |
  fn log(msg: &str) { println!("{}", msg); }

context: |
  pub counter: i32,

context_init: |
  counter: 0,

hooks:
  entry: |
    println!("Entering_{}", state_full_name);
  exit: |
    println!("Exiting_{}", state_full_name);

initial: active

states:
  active:
    initial: idle
    states:
      idle:
        transitions:
          - guard: ctx.counter >= 3
            to: working
      working:
        do: |
          ctx.counter += 1;
        transitions:
          - guard: ctx.counter >= 10
            to: null # Terminate

error:
  entry: |
    log("ERROR_STATE");
```

Listing 16: Complete SMB example (Rust)